Entrega D — Plano de monitoramento: SLIs, 5 métricas, alertas e limiares (Hora 5)

Regra de ouro 1: **definir o SLI antes do alerta.** Primeiro dizemos o que é "estar bom"; só depois criamos os alertas que avisam quando deixa de estar.

1. Três perguntas diferentes, três mecanismos diferentes

Verificação	Pergunta	Quem pergunta	Endpoint	Se falhar…	Implementado em
Liveness	O processo está vivo?	Docker (`healthcheck`)	`/health/live` (nunca atrasa, nunca depende de nada)	Docker marca `unhealthy`; reinício só se o processo morrer (`restart: on-failure:3`)	`docker-compose.yml`
Readiness	Pode receber tráfego agora?	Watchdog (5 s) e, indiretamente, o Nginx	`/health/ready` (200/503; aplica o atraso artificial)	Watchdog marca DOWN; o nó responde 503 em `/` e o Nginx o retira do pool **sem reiniciar**	`rust-app/src/main.rs`, `watchdog/watchdog.py`
Teste sintético	O caminho real do usuário funciona?	Watchdog (5 s) e `scripts/teste_sintetico.sh`	`GET http://nginx/` (passa pelo Nginx e por um nó)	Investigar a cadeia inteira (Nginx, nós, rede)	alvo `sintetico-nginx` do watchdog

Decisão de projeto importante (readiness → Nginx): o Nginx open source **não faz sondagem ativa** de `/health/ready`. Para a readiness ter efeito real no balanceamento, um nó que se declara "não pronto" responde **503** também em `GET /`. O Nginx trata 503 como falha (`proxy_next_upstream http_503`), refaz em outro nó e, após 3 falhas, retira o nó por 10 s. Evidência: `evidencias/preliminar/03_health_checks.txt` (`upstream_status=503, 200`).

2. SLIs e SLOs

SLI (indicador)	Como medir (PromQL)	SLO (meta) proposto	Por quê
Disponibilidade do caminho do usuário	sucesso do teste sintético: `avg_over_time(watchdog_target_up{target="sintetico-nginx"}[5m])`	≥ 99% em 5 min	Mede o que o usuário vê, não um nó isolado
Taxa de erro de usuário	`sum(rate(app_requests_total{route=~"/|/erro",status=~"5.."}[2m])) / sum(rate(app_requests_total{route=~"/|/erro"}[2m]))`	≤ 2% em 2 min	Abaixo disso é ruído para este volume

SLI (indicador)	Como medir (PromQL)	SLO (meta) proposto	Por quê
Latência p95 de `GET /`	`histogram_quantile(0.95, sum by (node,le) (rate(app_request_duration_seconds_bucket[2m])))`	p95 ≤ 1 s	1/3 do `proxy_read_timeout`: dá margem antes do retry

3. As 5 métricas escolhidas (+ complementares)

#	Métrica	Tipo	Origem	O que mede	Sinal clássico
M1	`watchdog_target_up{target,kind}` (e `watchdog_target_down`)	gauge 0/1	watchdog	Disponibilidade de cada nó **na visão de um observador externo** (3 falhas seguidas)	Disponibilidade
M2	`app_requests_total{node,route,status}`	counter	rust-a/b	Requisições por rota e status → taxa de erro 5xx e tráfego por nó (prova o balanceamento)	Taxa de erro / tráfego
M3	`app_request_duration_seconds_bucket{node,le}`	histogram	rust-a/b	Latência de `GET /` (inclui o atraso artificial) → p95	Latência
M4	`app_inflight_requests{node}`	gauge	rust-a/b	Requisições em andamento no nó	Saturação
M5	`watchdog_last_probe_timestamp_seconds{target}`	gauge (unix ts)	watchdog	Quando foi a última sondagem concluída → **idade da métrica / monitor vivo**	(saúde do monitor)

Complementares já expostas: `watchdog_consecutive_failures`, `watchdog_probe_duration_seconds`, `watchdog_probe_http_status`, `watchdog_probes_total{result}`, `watchdog_state_transitions_total`, `app_ready`, `app_artificial_delay_ms`, `app_start_time_seconds` (detecta reinício), `app_uptime_seconds`, e o `up{job}` do próprio Prometheus (métrica ausente).

Limitação declarada: o Nginx **não exporta métricas** (exigiria `nginx-prometheus-exporter`). Erros gerados **no próprio Nginx** (ex.: 502 quando os dois nós caem) não aparecem em M2; quem os vê é o teste sintético (M1, alvo `sintetico-nginx`) e o log de acesso do Nginx.

4. Alertas (arquivo `prometheus/alertas.yml`)

Avaliação a cada 5 s; coleta a cada 5 s. Sem Alertmanager: alertas visíveis em `http://127.0.0.1:9090/alerts` (limitação declarada).

4.1 Os três obrigatórios

Alerta	Condição e limiar	Justificativa do limiar	Resposta (procedimento)	Severidade / dono
NoDown	`watchdog_target_down{kind="readiness"} == 1` por 5 s	O watchdog só declara DOWN após **3 falhas × 5 s ≈ 15 s** → evita alarme por perda isolada; `for: 5s` exige mais uma coleta confirmando. Tempo total esperado: ~20–25 s	1) **Não reiniciar** de imediato. 2) Salvar evidências (`docker compose logs` , `ps`). 3) Testar o nó **de outro ponto** (`scripts/admin.sh check rust-a` , que parte de rust-b) e ver no log do Nginx se o nó ainda atende. 4) Só então retirar (`/admin/ready?v=0`) ou reiniciar	Alta / plantão
TaxaErro5xxAlta	5xx de usuário **> 2% em 2 min**, por 30 s	Abaixo de 2% (1 em 50) é ruído; acima o usuário percebe. Janela de 2 min suaviza picos; `for: 30s` evita disparo por rajada única. Sondas e `/metrics` ficam **fora** do denominador para não diluir a taxa	Ver por nó: `sum by (node,status) (rate(app_requests_total[2m]))` . Se é um nó: retirar do balanceamento. Se são os dois: bug comum, balanceamento não resolve — investigar a mudança recente	Alta / plantão
MonitorParado	`time() - max(max_over_time(watchdog_last_probe_timestamp_seconds[10m])) > 30`	30 s = **6 intervalos** sem sondar: não é atraso pontual. `max_over_time` mantém a expressão viva mesmo quando a série **some** (watchdog parado ⇒ coleta falha ⇒ série ausente). Folga para os saltos de relógio medidos no WSL2 (≤ 5,1 s)	**Salvar logs do watchdog e só então reiniciá-lo.** Enquanto parado, `NoDown` não é confiável: verificar nós manualmente	Média / equipe

4.2 Complementares

Alerta	Condição	Por quê	Severidade
TesteSinteticoFalhando	`watchdog_target_down{kind="sintetico"} == 1` por 5 s	O usuário já está sendo afetado (detecta Nginx caído ou os 2 nós fora)	Crítica
LatenciaP95Alta	p95 de `/` > 1 s por 1 min	Avisa antes de o Nginx começar a estourar o `proxy_read_timeout` (3 s) e fazer retry, que amplifica carga	Média
ColetaFalhando	`up{job=~"rust|watchdog"} == 0` por 15 s	**Métrica ausente**: separa "o nó caiu" de "o Prometheus não alcança o nó" quando comparado ao watchdog	Média
ReinicioEmLoop	`changes(app_start_time_seconds[10m]) > 2`	Um reinício é recuperação; três em 10 min é crash loop	Alta

Alerta	Condição	Por quê	Severidade
WatchdogFlapping	`changes(watchdog_target_up{kind="readiness"}[10m]) >= 6`	3 ciclos DOWN→UP em 10 min = nó instável ou rede intermitente. **Corrigido em 02/11**: a versão com `increase()` disparou com só 2 ciclos (extrapolação) — ver `docs/achados.md`	Média

5. Alerta sobre o próprio monitor — "quem vigia o vigia"

- O **watchdog** é vigiado pelo **Prometheus** (`MonitorParado`, `ColetaFalhando{job="watchdog"}`).
- O **Prometheus** não é vigiado por ninguém externo. É o SPOF residual, declarado.
- O **Docker** vigia a liveness de todos (healthcheck), mas não alerta ninguém: só muda o status em `docker compose ps`.

6. Exposição (passo 4 da Hora 5)

Endpoint	Acessível do host?	Evidência
`rust-*:8080/metrics`, `/admin/*`	Não: porta não publicada; e o Nginx devolve 404 para `/metrics` e `/admin/`	`evidencias/preliminar/09_portas_expostas.txt`, `03_health_checks.txt`
`watchdog:9100/metrics`	Não (porta não publicada)	`09_portas_expostas.txt`
Prometheus `:9090`	Só `127.0.0.1` (não acessível por outra máquina da rede)	`01_compose_ps.txt`